PART 9

Security

67 Secrets Management
68 IAM
69 OAuth
70 JWT Rotation
71 TLS
72 Encryption at Rest
73 Encryption in Transit
74 WAF
75 DDoS Protection
76 CORS
77 CSRF
78 SQL Injection
79 XSS
80 SSRF

#067

Secrets Management

WHAT IT IS

Storing and distributing credentials (API keys, DB passwords) securely.

WHY IT MATTERS

Leaked secrets are a top cause of breaches.

EXAMPLE

```
php artisan env:encrypt --env=production   #
    encrypted file safe to commit; key stored
    elsewhere
```

#068

IAM

WHAT IT IS

Identity and Access Management: who (identity) can do what (permissions) on which resource.

WHY IT MATTERS

Least privilege limits the blast radius of any compromise.

EXAMPLE

```
{ "Effect": "Allow", "Action": ["s3:GetObject",
    "s3:PutObject"],
  "Resource": "arn:aws:s3:::taskflow-uploads/tenants/
    *" }
```

#069

OAuth

WHAT IT IS

A protocol for delegated access: "let this app act on my behalf" without sharing my password (OAuth 2.0; OpenID Connect adds identity).

WHY IT MATTERS

"Login with Google/GitHub", API access for third parties.

EXAMPLE

```
// Laravel Socialite
return Socialite::driver('github')->redirect();

$ghUser = Socialite::driver('github')->user();
$user = User::updateOrCreate(['github_id' =>
    $ghUser->id], ['email' => $ghUser->email]);
```

#070

JWT Rotation

WHAT IT IS

Regularly replacing signing keys and issuing short-lived access tokens with refresh tokens.

WHY IT MATTERS

Limits damage when a token or key leaks; JWTs cannot be easily revoked once issued.

EXAMPLE

```
$payload = ['sub' => $user->id, 'iat' => time(),
    'exp' => time() + 900];
JWT::encode($payload, $privateKey, 'RS256', keyId:
    'key-2026-10');
```

#071

TLS

WHAT IT IS

The protocol behind HTTPS: encrypts traffic and authenticates the server.

WHY IT MATTERS

Stops eavesdropping and tampering; required by browsers and compliance.

EXAMPLE

```
ssl_protocols TLSv1.2 TLSv1.3;
add_header Strict-Transport-Security
    "max-age=31536000; includeSubDomains" always;
```

#072

Encryption at Rest

WHAT IT IS

Data is encrypted while stored on disk/DB/backups.

WHY IT MATTERS

Protects from stolen disks, leaked backups, and some insider risks.

EXAMPLE

```
protected function casts(): array {
    return ['tax_id' => 'encrypted', 'api_token' =>
    'encrypted'];
}
```

#073

Encryption in Transit

WHAT IT IS

Data is encrypted while moving between systems.

WHY IT MATTERS

Internal networks are not automatically safe.

EXAMPLE

```
DB_SSLMODE=verify-full
REDIS_SCHEME=tls
```

#074

WAF

WHAT IT IS

Web Application Firewall: filters malicious HTTP traffic (SQLi, XSS, bad bots) before it reaches your app.

WHY IT MATTERS

Cheap, fast protection layer and virtual patching during incidents.

EXAMPLE

Cloudflare WAF, AWS WAF, ModSecurity + OWASP Core Rule Set. Rule: block requests where a query string matches SQLi patterns; rate-limit /login per IP. A WAF complements, not replaces, secure code.

#075

DDoS Protection

WHAT IT IS

Defences against floods of traffic meant to knock you offline.

WHY IT MATTERS

Availability is a security property.

EXAMPLE

Put a CDN/proxy in front (Cloudflare, AWS Shield), hide origin IP, enable rate limiting and bot challenges, cache aggressively, auto-scale, and restrict the origin firewall to the CDN's IP ranges only. Application-layer floods (expensive endpoints) need per-route limits too.

#076

CORS

WHAT IT IS

Browser rule controlling which other origins may call your API from JavaScript.

WHY IT MATTERS

Misconfiguration (* with credentials) exposes user data; too strict breaks your frontend.

EXAMPLE

```
// config/cors.php
'paths' => ['api/*'],
'allowed_origins' => ['https://app.taskflow.com'],
'allowed_methods' => ['GET', 'POST', 'PUT',
    'DELETE'],
'supports_credentials' => true,
```

#077

CSRF

WHAT IT IS

Cross-Site Request Forgery: a malicious site makes the victim's browser send an authenticated request to your site.

WHY IT MATTERS

Cookie-based sessions are sent automatically, enabling unwanted actions.

EXAMPLE

```
<form method="POST" action="/tasks/1/delete">
    @csrf
    <button>Delete</button>
</form>
```

#078

SQL Injection

WHAT IT IS

Attacker input is interpreted as SQL.

WHY IT MATTERS

Can leak or destroy the entire database.

EXAMPLE

```
// Vulnerable
DB::select("SELECT * FROM users WHERE email =
    '$email'");

// Safe: bindings
DB::select('SELECT * FROM users WHERE email = ?',
    [$email]);
User::where('email', $email)->first();

// Careful with raw column names/order by: whitelist
    them
$col = in_array($req->sort, ['title', 'due_at']) ?
    $req->sort : 'id';
```

#079

XSS

WHAT IT IS

Cross-Site Scripting: attacker-supplied script runs in another user's browser.

WHY IT MATTERS

Steals sessions, performs actions as the victim.

EXAMPLE

```
{{ $comment->body }}      {{-- escaped, safe --}}
{!! $comment->body !!}    {{-- raw, dangerous unless
    sanitized --}}
```

#080

SSRF

WHAT IT IS

Server-Side Request Forgery: attacker makes your server fetch internal URLs (cloud metadata, admin panels).

WHY IT MATTERS

Can expose cloud credentials (http://169.254.169.254/...).

EXAMPLE

```
$host = parse_url($url, PHP_URL_HOST);
$ip = gethostbyname($host);
if (! filter_var($ip, FILTER_VALIDATE_IP,
    FILTER_FLAG_NO_PRIV_RANGE |
    FILTER_FLAG_NO_RES_RANGE)) {
    abort(422, 'URL not allowed');
}
Http::withoutRedirecting()->timeout(5)->post($url,
    $payload); // also re-check after DNS to avoid
    rebinding
```

Found this useful?

Save it for later.

Share it with a teammate.

Full guide with all code examples:
the Backend Engineering Handbook (GitHub).